

INFORMATION SECURITY



Rishabh Rathore

B.Sc. (H.)-Computer Science (VIII sem)

16139

Practical File

P1). Demonstrate the use of Network tools: ping, ipconfig, ifconfig, traceroute, arp, netstat, Whois.

1. Ping - Check connectivity to a host

```
(Rishabh@HackingMachine)-[~]
$ ping -c 4 google.com
PING google.com (142.250.76.78) 56(84) bytes of data:
64 bytes from maa05s14-in-f14.1e100.net (142.250.76.78): icmp_seq=1 ttl=255 time=3.16 ms
64 bytes from maa05s14-in-f14.1e100.net (142.250.76.78): icmp_seq=2 ttl=255 time=5.05 ms
64 bytes from maa05s14-in-f14.1e100.net (142.250.76.78): icmp_seq=3 ttl=255 time=3.31 ms
64 bytes from maa05s14-in-f14.1e100.net (142.250.76.78): icmp_seq=4 ttl=255 time=6.33 ms

— google.com ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3253ms
rtt min/avg/max/mdev = 3.155/4.460/6.326/1.310 ms
```

2. ifconfig (Linux) - Show your IP, subnet, gateway.

```
(Rishabh@HackingMachine)-[~]
$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether be:dc:de:69:29:7d txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 6 overruns 0 carrier 0 collisions 0
```

3. traceroute (Linux) - Show route to a remote host.

```
(Rishabh@HackingMachine)-[~]
$ traceroute google.com
traceroute to google.com (142.250.76.78), 30 hops max, 60 byte packets
 1  10.0.2.2 (10.0.2.2)  2.740 ms  2.617 ms  2.547 ms
 2  * * *
 3  * * *
 4  * * *
```

4. arp - Show ARP table.

```
(Rishabh@HackingMachine)-[~]
$ arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
192.168.56.1              ether   0a:00:27:00:00:04    C                     eth1
10.0.2.2                  ether   52:55:0a:00:02:02    C                     eth0
192.168.56.100            ether   08:00:27:b3:38:1a    C                     eth1
```

5. Netstat - Show active connections and listening ports.

```
(Rishabh@HackingMachine)-[~]
$ netstat -tulnp
(Not all processes could be identified, non-owned process info
will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:35671         0.0.0.0:*                LISTEN      -
tcp        0      0 127.0.0.1:5432         0.0.0.0:*                LISTEN      -
tcp        0      0 127.0.0.1:9392         0.0.0.0:*                LISTEN      -
tcp        0      0 127.0.0.1:1883         0.0.0.0:*                LISTEN      -
tcp6       0      0 :::5432                 :::*                    LISTEN      -
tcp6       0      0 :::1883                 :::*                    LISTEN      -
```

6. Whois - Show domain registration info

```
(Rishabh@HackingMachine)-[~]
$ whois google.com
Domain Name: GOOGLE.COM
Registry Domain ID: 2138514_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Updated Date: 2019-09-09T15:39:04Z
Creation Date: 1997-09-15T04:00:00Z
Registry Expiry Date: 2028-09-14T04:00:00Z
Registrar: MarkMonitor Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrar Abuse Contact Phone: +1.2086851750
Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited
Domain Status: serverDeleteProhibited https://icann.org/epp#serverDeleteProhibited
Domain Status: serverTransferProhibited https://icann.org/epp#serverTransferProhibited
Domain Status: serverUpdateProhibited https://icann.org/epp#serverUpdateProhibited
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-05-13T16:11:00Z <<<
```

“These tools help in troubleshooting connectivity, viewing routing, mapping IPs to MAC addresses, and checking open ports.”

P2). Use of Password cracking tools: John the Ripper, Ophcrack. Verify the strength of passwords using these tools.

```
(Rishabh@HackingMachine)-[~]
$ echo "rishabh:$(openssl passwd -1 weakpass123)" > testpass.txt

(Rishabh@HackingMachine)-[~]
$ john testpass.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8x3])
Will run 4 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
Proceeding with incremental:ASCII
0g 0:00:00:26 3/3 0g/s 321150p/s 321150c/s 321150C/s minkroxx..minnys23
0g 0:00:00:26 3/3 0g/s 323225p/s 323225c/s 323225C/s midlo..meagh
0g 0:00:02:22 3/3 0g/s 237867p/s 237867c/s 237867C/s dd6..lmorg7
0g 0:00:02:32 3/3 0g/s 248933p/s 248933c/s 248933C/s abdrys131..1til5
0g 0:00:04:28 3/3 0g/s 281598p/s 281598c/s 281598C/s 3h1..samappan
0g 0:00:05:08 3/3 0g/s 295755p/s 295755c/s 295755C/s sandiott..sanic135
0g 0:00:08:05 3/3 0g/s 283411p/s 283411c/s 283411C/s sourang23..sourash10
0g 0:00:09:02 3/3 0g/s 295254p/s 295254c/s 295254C/s 0876771963..0873031088
0g 0:00:09:43 3/3 0g/s 301873p/s 301873c/s 301873C/s eackob..eac95l
Session aborted
```

P3). Use nmap/zenmap to analyze a remote machine.

```
(Rishabh@HackingMachine)-[~]
$ nmap 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-13 21:54 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000020s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
5432/tcp  open  postgresql

Nmap done: 1 IP address (1 host up) scanned in 0.07 seconds

(Rishabh@HackingMachine)-[~]
$ nmap -sV 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-13 21:54 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000020s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
5432/tcp  open  postgresql  PostgreSQL DB 18.0 - 18.2

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.28 seconds

(Rishabh@HackingMachine)-[~]
$ nmap -A 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-13 21:55 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000094s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
5432/tcp  open  postgresql  PostgreSQL DB 18.0 - 18.2
|_ ssl-cert: Subject: commonName=debos-2460130654074344449
|_ Subject Alternative Name: DNS:debos-2460130654074344449
|_ Not valid before: 2026-03-20T06:43:59
|_ Not valid after: 2036-03-17T06:43:59
|_ tls-alpn:
|_ postgresql
|_ ssl-date: TLS randomness does not represent time
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.99%E=4%D=5/13%OT=5432%CT=1%CU=33661%PV=Y%DS=0%DC=L%G=Y%TM=6A04A
OS:5FB%P=x86_64-pc-linux-gnu)SEQ(SP=102%GCD=1%ISR=10B%TI=Z%CI=Z%II=I%TS=22)
OS:SEQ(SP=102%GCD=1%ISR=10C%TI=Z%CI=Z%II=I%TS=21)SEQ(SP=103%GCD=1%ISR=10E%T
OS:I=Z%CI=Z%II=I%TS=21)SEQ(SP=104%GCD=1%ISR=109%TI=Z%CI=Z%II=I%TS=22)SEQ(SP
OS:=107%GCD=1%ISR=10A%TI=Z%CI=Z%II=I%TS=21)OPS(O1=MFFD7ST11NWA%O2=MFFD7ST11
OS:NWA%O3=MFFD7NNT11NWA%O4=MFFD7ST11NWA%O5=MFFD7ST11NWA%O6=MFFD7ST11)WIN(W1
OS:=FFCB%W2=FFCB%W3=FFCB%W4=FFCB%W5=FFCB%W6=FFCB)ECN(R=Y%DF=Y%T=40%W=FFD7%O
OS:=MFFD7NNSNWA%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=0%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=
OS:N)T4(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A
OS:=S+%F=AR%O=%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T7(R=Y%D
OS:F=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL
OS:=G%RID=G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)
```

P4). Use Burp proxy to capture and modify the message.

1. Starting of the port

```
(Rishabh@HackingMachine)-[~]
$ mkdir -p ~/burp-test

(Rishabh@HackingMachine)-[~]
$ cd ~/burp-test

(Rishabh@HackingMachine)-[~/burp-test]
$ echo "Hello from Rishabh" > index.html

(Rishabh@HackingMachine)-[~/burp-test]
$ python3 -m http.server 800
Serving HTTP on 0.0.0.0 port 800 (http://0.0.0.0:800/) ...
^C
Keyboard interrupt received, exiting.

(Rishabh@HackingMachine)-[~/burp-test]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [13/May/2026 22:12:17] "GET /?name=rishabh HTTP/1.1" 200 -Tab
```

2. Installing the burpsuite

```
(Rishabh@HackingMachine)-[~/Downloads]
$ chmod +x burpsuite_linux_v2026_4_3.sh

(Rishabh@HackingMachine)-[~/Downloads]
$ ./burpsuite_linux_v2026_4_3.sh
Unpacking JRE ...
Starting Installer ...

(Rishabh@HackingMachine)-[~/Downloads]
$
```

3. Intercepted request

The screenshot displays the Burp Suite interface with the 'Proxy' tab selected. The 'Intercept' tab is active, showing a list of intercepted requests. The second request, at 22:12:00, is selected and its details are shown in the 'Request' and 'Inspector' panels.

Request List:

Time	Type	Direction	Method	URL	Status code	Length
22:11:15...	HTTP	→ Request	GET	http://localhost:8000/		
22:12:00...	HTTP	→ Request	GET	http://localhost:8000/?name=admin		

Request Details (Pretty view):

```
1 GET /?name=admin HTTP/1.1
2 Host: localhost:8000
3 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Linux"
6 Accept-Language: en-US,en;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/146.0.0.0 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/
  apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: none
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
```

Inspector:

- Request attributes: 2
- Request query parameters: 1
- Request body parameters: 0
- Request cookies: 0
- Request headers: 14

Event log: All issues

Memory: 164.4MB of 2.89GB

Disabled: [v]

4. Intercepted message

```
(Rishabh@HackingMachine)-[~/burp-test]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [13/May/2026 22:12:17] "GET /?name=rishabh HTTP/1.1" 200 -
```

- **Aim:** Use Burp proxy to capture and modify HTTP messages.
- **Procedure:**
 - Start local server with `python3 -m http.server 8000`.
 - Start Burp, enable Proxy → Intercept.
 - Configure browser proxy to 127.0.0.1:8080.
 - Access <http://localhost:8000/?name=rishabh>.
 - Modify name=rishabh to name=admin in Burp and forward.
- **Observation:** The server log shows `/index.html?name=admin`.
- **Result:** Demonstrates that an intercepting proxy can view and alter HTTP requests in transit.

P5). Implement caesar cipher substitution operation.

P6). Implement monoalphabetic and polyalphabetic cipher substitution operation.

P7). Implement playfair cipher substitution operation.

P8). Implement hill cipher substitution operation.

P9). Implement rail fence cipher transposition operation.

P10). Implement row transposition cipher transposition operation.

P11). Implement product cipher transposition operation.

***“All of these ciphers are done in python notebook
attached in a single notebook below”***

```

import math
import numpy as np

def hill_cipher(text, key_matrix):
    text = text.upper().replace(" ", "")
    n = len(key_matrix)

    if len(text) % n != 0:
        text += 'X' * (n - (len(text) % n))

    vector = [ord(char) - 65 for char in text]
    vector = np.array(vector).reshape(-1, n)

    result = ""
    for row in vector:
        encrypted_row = np.dot(key_matrix, row) % 26
        result += "".join([chr(int(num) + 65) for num in encrypted_row])
    return result

def rail_fence(text, rails):
    text = text.replace(" ", "")
    if rails == 1: return text

    fence = ["" for _ in range(rails)]
    rail_index, direction = 0, 1

    for char in text:
        fence[rail_index] += char
        rail_index += direction
        if rail_index == 0 or rail_index == rails - 1:
            direction *= -1

    return "".join(fence)

def rail_fence_decrypt(cipher_text, rails):
    if rails == 1: return cipher_text

    rail_matrix = [['\n' for _ in range(len(cipher_text))] for _ in range(rails)]
    row, col = 0, 0
    direction = 1

    for _ in range(len(cipher_text)):
        rail_matrix[row][col] = '*'
        col += 1
        row += direction
        if row == 0 or row == rails - 1:
            direction *= -1

```

```

index = 0
for r in range(rails):
    for c in range(len(cipher_text)):
        if rail_matrix[r][c] == '*':
            rail_matrix[r][c] = cipher_text[index]
            index += 1

result = []
row, col = 0, 0
direction = 1

for _ in range(len(cipher_text)):
    result.append(rail_matrix[row][col])
    col += 1
    row += direction
    if row == 0 or row == rails - 1:
        direction *= -1

return "".join(result)

def row_transposition(text, key):
    text = text.replace(" ", "")
    key_order = sorted(list(enumerate(key)), key=lambda x: x[1])

    col_count = len(key)
    row_count = math.ceil(len(text) / col_count)
    padded_text = text.ljust(row_count * col_count, 'X')

    grid = [padded_text[i:i + col_count] for i in range(0,
len(padded_text), col_count)]
    ciphertext = ""
    for original_index, char_val in key_order:
        for row in grid:
            ciphertext += row[original_index]

    return ciphertext

def row_transposition_decrypt(cipher_text, key):
    col_count = len(key)
    row_count = math.ceil(len(cipher_text) / col_count)
    key_order = sorted(list(enumerate(key)), key=lambda x: x[1])

    grid = [['' for _ in range(col_count)] for _ in range(row_count)]

    col_length = row_count

    current_cipher_index = 0
    for original_col_index, _ in key_order:
        for r in range(row_count):
            grid[r][original_col_index] =

```

```

cipher_text[current_cipher_index]
    current_cipher_index += 1

plaintext = ""
for r in range(row_count):
    for c in range(col_count):
        plaintext += grid[r][c]

return plaintext.rstrip('X')

def caesar_cipher(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            start = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char) - start + shift) % 26 + start)
        else:
            result += char
    return result

def caesar_decrypt(text, shift):
    return caesar_cipher(text, -shift)

def product_cipher(text, shift, rails):
    substituted = caesar_cipher(text, shift)
    return rail_fence(substituted, rails)

def product_cipher_decrypt(cipher_text, shift, rails):
    decrypted_rail = rail_fence_decrypt(cipher_text, rails)
    decrypted_caesar = caesar_decrypt(decrypted_rail, shift)
    return decrypted_caesar

```

Cipher Operations Menu

```

def run_cipher_menu():
    while True:
        print("\n--- Cipher Operations ---")
        print("1. Encrypt Message")
        print("2. Decrypt Message")
        print("3. Exit")

        choice = input("Enter your choice (1-3): ")

        if choice == '3':
            print("Exiting program. Goodbye!")
            break

        if choice in ['1', '2']:
            operation_type = "encrypt" if choice == '1' else "decrypt"

```

```

        while True:
            print(f"\n--- Choose a Cipher to
{operation_type.capitalize()} ---")
            print("1. Caesar Cipher")
            print("2. Rail Fence Cipher")
            print("3. Row Transposition Cipher")
            print("4. Product Cipher (Caesar + Rail Fence)")
            print("5. Hill Cipher (Encryption Only)")
            print("6. Back to Main Menu")

            cipher_choice = input("Enter your cipher choice (1-6):
")

            if cipher_choice == '6':
                break

            text = input(f"Enter the text to {operation_type}: ")
            result = ""
            valid_input = True

            if cipher_choice == '1':
                try:
                    shift = int(input("Enter the shift value (an
integer): "))

                    if operation_type == "encrypt":
                        result = caesar_cipher(text, shift)
                    else:
                        result = caesar_decrypt(text, shift)
                except ValueError:
                    print("Invalid shift value. Please enter an
integer.")

                    valid_input = False
            elif cipher_choice == '2':
                try:
                    rails = int(input("Enter the number of rails
(an integer > 1): "))

                    if rails <= 1:
                        print("Number of rails must be greater
than 1.")

                        valid_input = False
                    else:
                        if operation_type == "encrypt":
                            result = rail_fence(text, rails)
                        else:
                            result = rail_fence_decrypt(text,
rails)

                except ValueError:
                    print("Invalid number of rails. Please enter
an integer.")

                    valid_input = False

```

```

        elif cipher_choice == '3':
            key = input("Enter the key (a string, e.g.,
'KEY'): ")
            if not key:
                print("Key cannot be empty.")
                valid_input = False
            else:
                if operation_type == "encrypt":
                    result = row_transposition(text, key)
                else:
                    result = row_transposition_decrypt(text,
key)
            elif cipher_choice == '4':
                try:
                    shift = int(input("Enter the Caesar shift
value (an integer): "))
                    rails = int(input("Enter the number of rails
for Rail Fence (an integer > 1): "))
                    if rails <= 1:
                        print("Number of rails must be greater
than 1.")
                        valid_input = False
                    else:
                        if operation_type == "encrypt":
                            result = product_cipher(text, shift,
rails)
                        else:
                            result = product_cipher_decrypt(text,
shift, rails)
                except ValueError:
                    print("Invalid shift or rails value. Please
enter integers.")
                    valid_input = False
            elif cipher_choice == '5':
                if operation_type == "decrypt":
                    print("Decryption for Hill Cipher is not
implemented in this menu.")
                    valid_input = False
                else:
                    try:
                        key_str = input("Enter the key matrix
(e.g., '[[5,8],[17,3]]' for a 2x2 matrix): ")
                        key_matrix = np.array(eval(key_str))
                        if key_matrix.shape[0] !=
key_matrix.shape[1]:
                            print("Key matrix must be square.")
                            valid_input = False
                        else:
                            result = hill_cipher(text, key_matrix)

```

```

                                except (ValueError, SyntaxError, NameError) as
e:
                                print(f"Invalid key matrix format: {e}.
Please ensure it's a valid nested list representation.")
                                valid_input = False
                                else:
                                print("Invalid cipher choice. Please try again.")
                                valid_input = False

                                if valid_input and result:
                                print(f"\n{'Encrypted' if operation_type ==
'encrypt' else 'Decrypted'} message: {result}")
                                elif valid_input and not result:
                                pass

                                else:
                                print("Invalid main menu choice. Please enter 1, 2, or
3.")

run_cipher_menu()

--- Cipher Operations ---
1. Encrypt Message
2. Decrypt Message
3. Exit
Enter your choice (1-3): 3
Exiting program. Goodbye!

import numpy as np

def get_key():
    return input("Enter a key : ")

alpha_list_upper = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

def remove_duplicate_from_key(key):
    final_key = ""
    for char in key:
        if char not in final_key:
            final_key += char
    return final_key

key = get_key()
key = key.upper().replace("J", "I")
key = remove_duplicate_from_key(key)

key_matrix = [['' for _ in range(5)] for _ in range(5)]

def create_final_matrix():
    inserted_elements = list(key)

```

```

    for alpha in alpha_list_upper:
        if alpha not in inserted_elements:
            inserted_elements.append(alpha)

    count = 0
    for i in range(5):
        for j in range(5):
            key_matrix[i][j] = inserted_elements[count]
            count += 1

    return key_matrix

key_matrix = create_final_matrix()

def print_matrix():
    print("\nPlayfair Matrix:")
    for row in key_matrix:
        print(" ".join(row))

print_matrix()

def preprocess_text(text):
    text = text.upper().replace("J", "I").replace(" ", "")
    pairs = []
    i = 0

    while i < len(text):
        a = text[i]
        if i+1 < len(text):
            b = text[i+1]
            if a == b:
                pairs.append(a + "X")
                i += 1
            else:
                pairs.append(a + b)
                i += 2
        else:
            pairs.append(a + "X")
            i += 1

    return pairs

def find_position(char):
    for i in range(5):
        for j in range(5):
            if key_matrix[i][j] == char:
                return i, j

```

```

def encrypt(plaintext):
    pairs = preprocess_text(plaintext)
    cipher = ""

    for pair in pairs:
        a, b = pair[0], pair[1]
        r1, c1 = find_position(a)
        r2, c2 = find_position(b)

        if r1 == r2:
            cipher += key_matrix[r1][(c1+1)%5]
            cipher += key_matrix[r2][(c2+1)%5]

        elif c1 == c2:
            cipher += key_matrix[(r1+1)%5][c1]
            cipher += key_matrix[(r2+1)%5][c2]

        else:
            cipher += key_matrix[r1][c2]
            cipher += key_matrix[r2][c1]

    return cipher

def decrypt(ciphertext):
    pairs = preprocess_text(ciphertext)
    plain = ""

    for pair in pairs:
        a, b = pair[0], pair[1]
        r1, c1 = find_position(a)
        r2, c2 = find_position(b)

        if r1 == r2:
            plain += key_matrix[r1][(c1-1)%5]
            plain += key_matrix[r2][(c2-1)%5]

        elif c1 == c2:
            plain += key_matrix[(r1-1)%5][c1]
            plain += key_matrix[(r2-1)%5][c2]

        else:
            plain += key_matrix[r1][c2]
            plain += key_matrix[r2][c1]

    return plain

text = input("\nEnter plaintext: ")

```

```
cipher = encrypt(text)
print("Encrypted:", cipher)

decrypted = decrypt(cipher)
print("Decrypted:", decrypted)
```